

WebSocket Events Mapping: Client ↔ Server

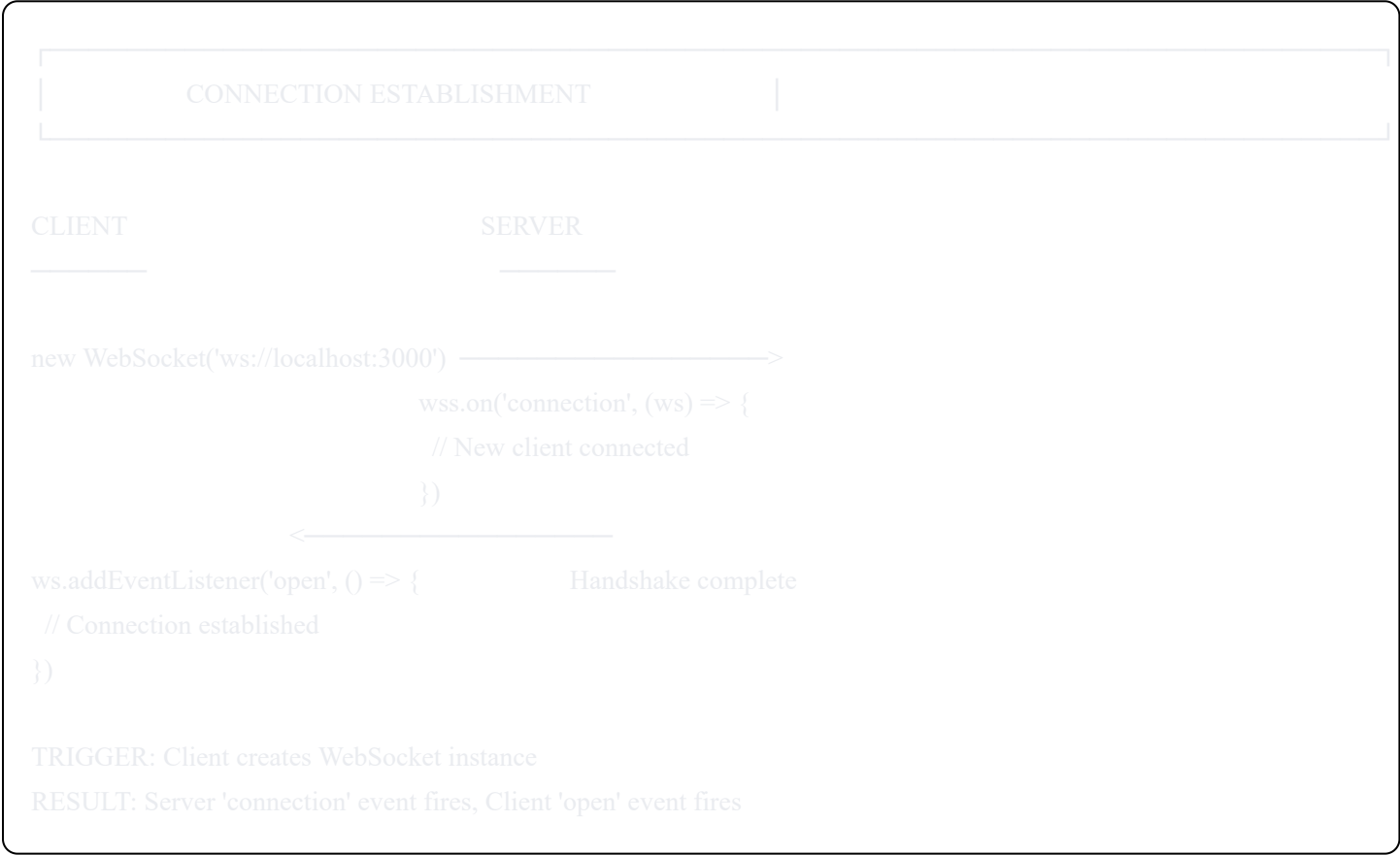
Complete Event Correspondence Map

This document shows the complete mapping between client-side and server-side WebSocket events, explaining exactly how they trigger each other.

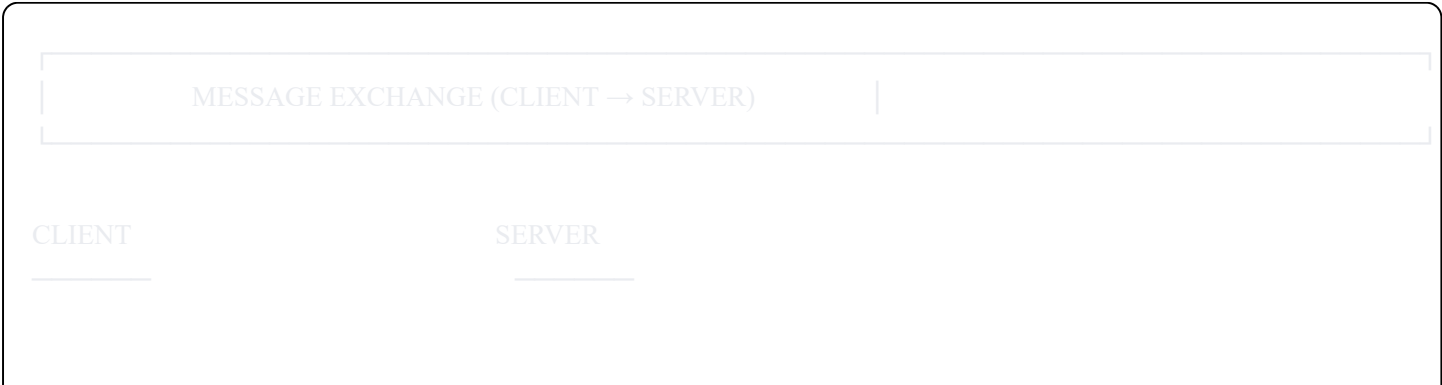
1. Native WebSocket Events (Built into Protocol)

These are the core events that are **built into the WebSocket protocol** and handled automatically by the WebSocket library/browser.

Event Map: Connection Establishment



Event Map: Message Exchange



```
ws.send('Hello')  
  
ws.on('message', (data) => {  
  console.log(data); // 'Hello'  
})
```

TRIGGER: Client calls ws.send()

RESULT: Server 'message' event fires with data

MESSAGE EXCHANGE (SERVER → CLIENT)

CLIENT

SERVER

```
ws.addListener('message', (e) => {  
  console.log(e.data); // 'Hello from server'  
})  
  
ws.send('Hello from server')
```

TRIGGER: Server calls ws.send()

RESULT: Client 'message' event fires with data in event.data

Event Map: Connection Close

CONNECTION CLOSE (CLIENT INITIATED)

CLIENT

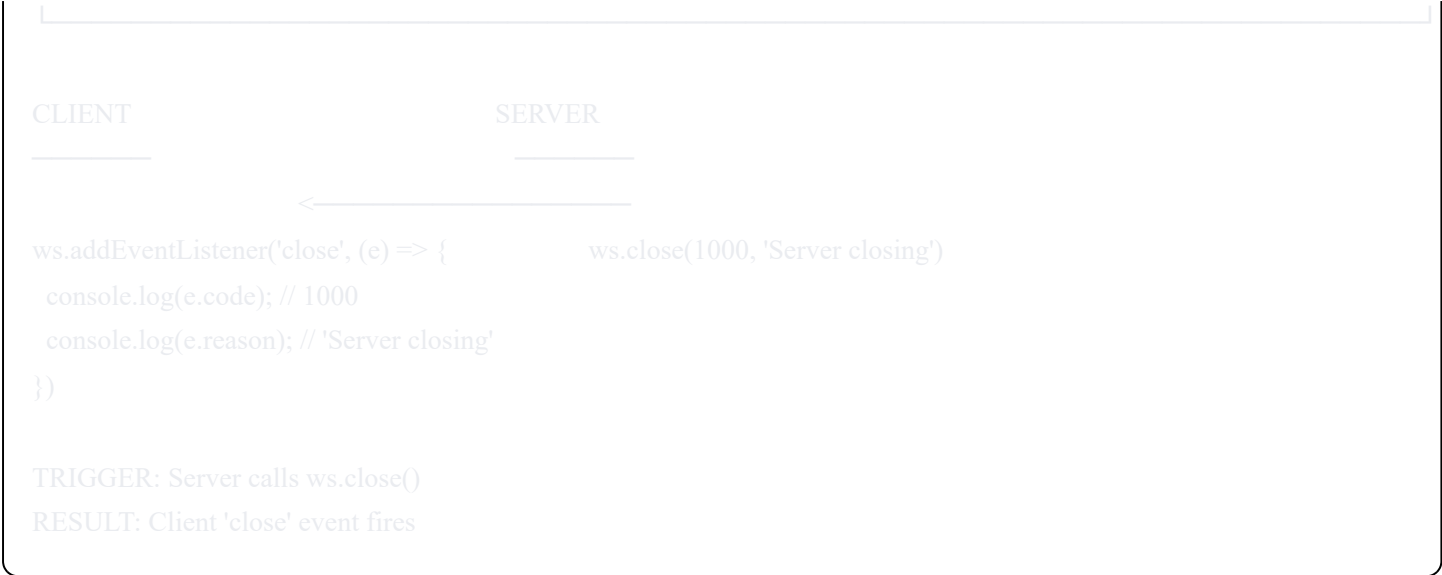
SERVER

```
ws.close(1000, 'Goodbye')  
  
ws.on('close', (code, reason) => {  
  console.log(code); // 1000  
  console.log(reason); // 'Goodbye'  
})  
  
ws.addListener('close', (e) => {  
  console.log(e.code); // 1000  
})
```

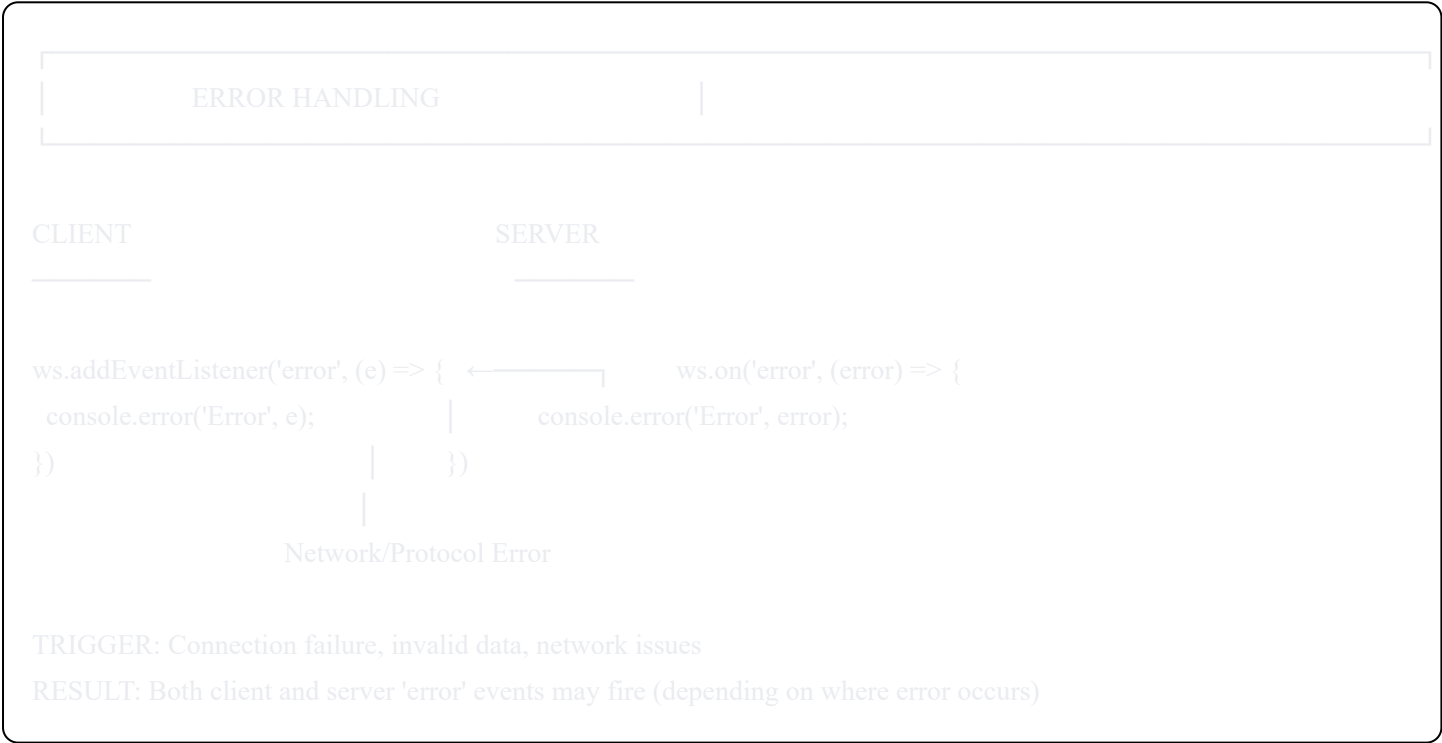
TRIGGER: Client calls ws.close()

RESULT: Server 'close' event fires, then Client 'close' event fires

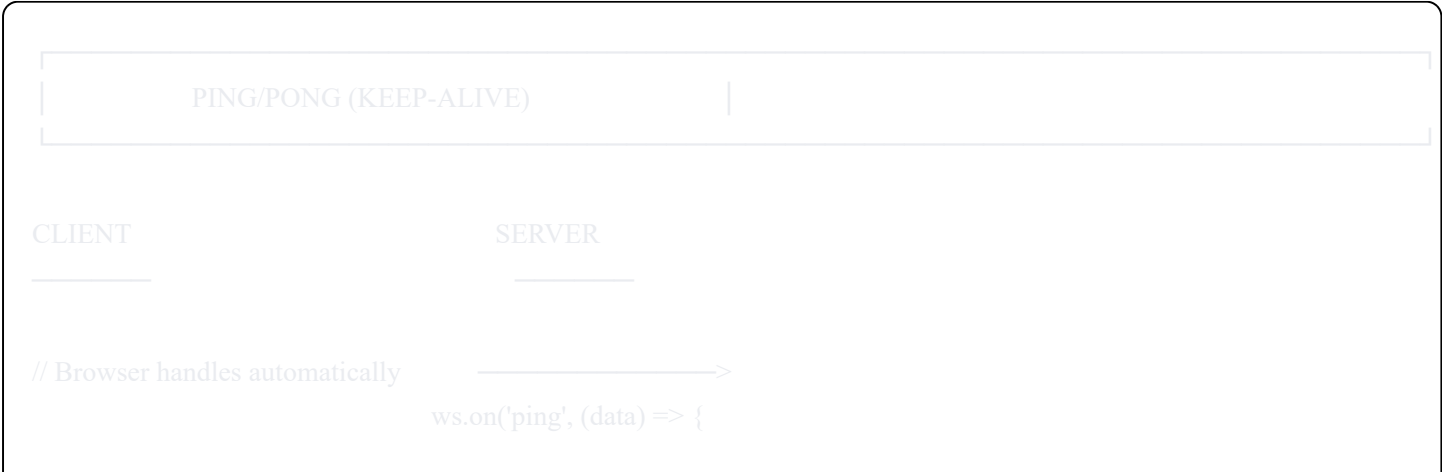
CONNECTION CLOSE (SERVER INITIATED)



Event Map: Errors



Event Map: Ping/Pong (Heartbeat)



// Ping received
})

←

// Browser handles automatically
ws.on('pong', (data) => {
// Pong received
})

// Server initiating ping

←

// Browser auto-responds with pong
ws.ping()

TRIGGER: Either side can send ping, other side auto-responds with pong
RESULT: Keeps connection alive, detects dead connections
NOTE: Browser WebSocket API handles ping/pong automatically, not exposed to JavaScript

2. Application-Level Events (Custom Message Types)

These are **custom events** we implement using the `message` event by structuring our data with a `type` field.

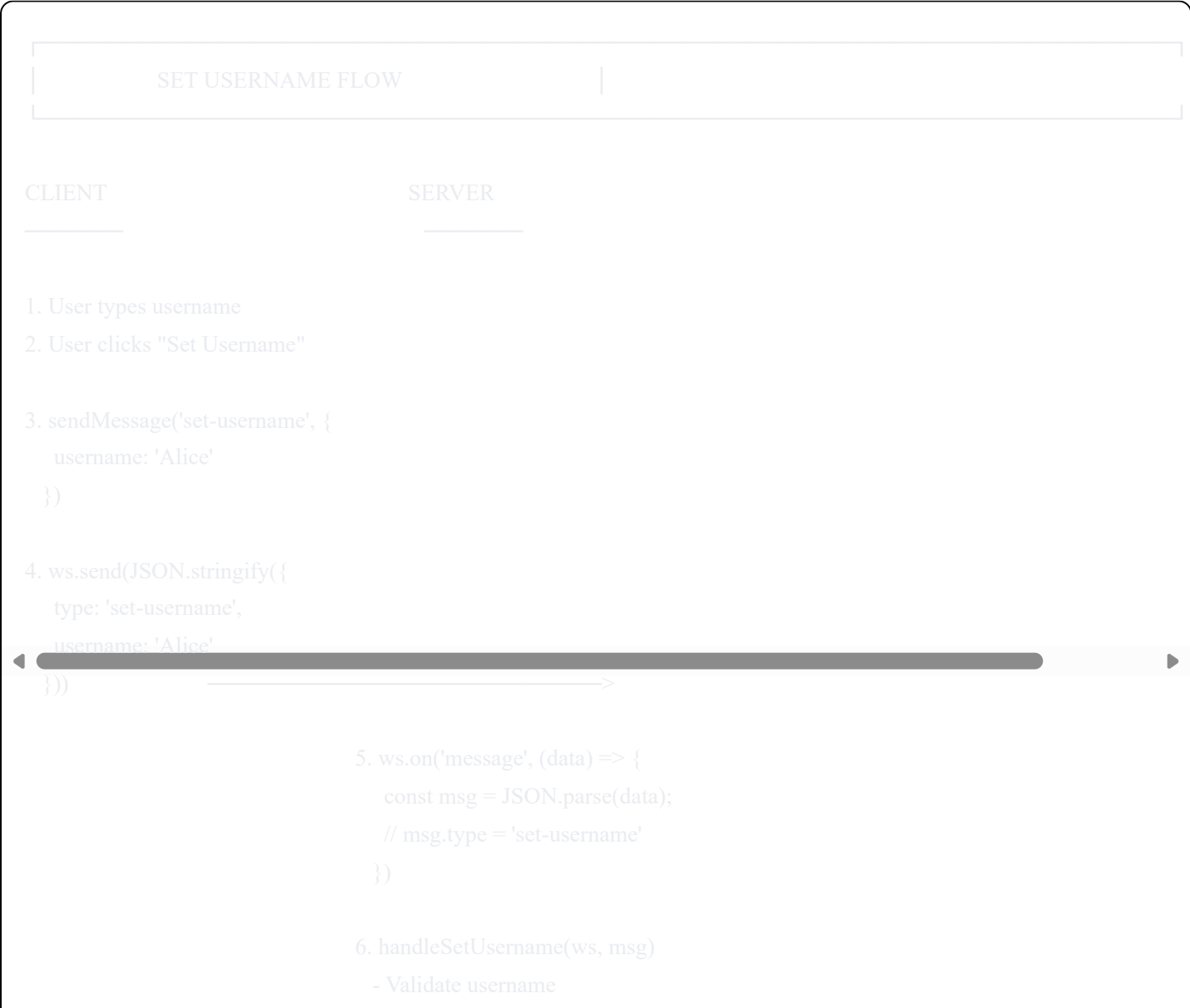
Complete Message Type Mapping

APPLICATION EVENT MAPPING		
CLIENT ACTION	MESSAGE SENT	SERVER HANDLER
Set Username	type: 'set-username'	handleSetUsername()
Join Room	type: 'join-room'	handleJoinRoom()
Leave Room	type: 'leave-room'	handleLeaveRoom()
Send Chat Message	type: 'chat'	handleChatMessage()
Create Room	type: 'create-room'	handleCreateRoom()
Send Private Message	type: 'private-message'	handlePrivateMessage()
Typing Indicator	type: 'typing'	handleTyping()
Request Room List	type: 'get-rooms'	handleGetRooms()
Request User List	type: 'get-users'	handleGetUsers()
SERVER ACTION	MESSAGE SENT	CLIENT HANDLER
Connection Established	type: 'connection'	handleConnection()
Username Confirmed	type: 'username-set'	handleUsernameSet()

Room Joined Confirmation	type: 'joined-room'	handleJoinedRoom()
Room Left Confirmation	type: 'left-room'	handleLeftRoom()
Broadcast Chat Message	type: 'chat'	handleChatMessage()
User Joined Room (broadcast)	type: 'user-joined'	handleUserJoined()
User Left Room (broadcast)	type: 'user-left'	handleUserLeft()
Room Members List	type: 'room-members'	handleRoomMembers()
Room Created (broadcast)	type: 'room-created'	handleRoomCreated()
Private Message Received	type: 'private-message'	handlePrivateMessage()
Private Message Sent Confirm	type: 'private-message-sent'	handlePrivateMessageSent()
Someone is Typing	type: 'typing'	handleTyping()
Error Message	type: 'error'	handleError()
Available Rooms	type: 'rooms-list'	handleRoomsList()
Online Users	type: 'users-list'	handleUsersList()

3. Detailed Event Flow Diagrams

Flow 1: User Sets Username



- Check if taken
- Store in clients Map

```
7. ws.send(JSON.stringify({  
  type: 'username-set',  
  username: 'Alice'  
}))
```

```
8. ws.addEventListener('message', (e) => {  
  const data = JSON.parse(e.data);  
  // data.type = 'username-set'  
})
```

```
9. handleUsernameSet(data)  
- Store username locally  
- Show chat interface  
- Hide username form
```

Flow 2: User Joins Room

JOIN ROOM FLOW

CLIENT (Alice)

SERVER

1. User clicks room "general"

```
2. sendMessage('join-room', {  
  room: 'general'  
})
```

```
3. ws.send(JSON.stringify({  
  type: 'join-room',  
  room: 'general'  
}))
```

```
4. ws.on('message', (data) => {  
  // Receives join-room  
})
```

```
5. handleJoinRoom(ws, msg)  
- Add to rooms Map
```

- Get room members

```
6. ws.send(JSON.stringify({  
  type: 'joined-room',  
  room: 'general'  
}))
```

7. handleJoinedRoom(data)

- Update UI
- Clear messages
- Show room name

```
8. broadcastToRoom('general', {  
  type: 'user-joined',  
  username: 'Alice'  
}, excludeAlice)
```

CLIENT (Bob - already in room)

```
9. ws.addEventListener('message', (e) => {  
  const data = JSON.parse(e.data);  
  // data.type = 'user-joined'  
})
```

10. handleUserJoined(data)

- Show "Alice joined the room"

```
11. ws.send(JSON.stringify({  
  type: 'room-members',  
  members: ['Alice', 'Bob']  
}))
```

CLIENT (Alice)

12. handleRoomMembers(data)

- Display member list

Flow 3: User Sends Chat Message

SEND CHAT MESSAGE FLOW

CLIENT (Alice)

SERVER

- 1. User types "Hello everyone!"
- 2. User presses Enter or clicks Send
- 3. `sendChatMessage()`

```
4. ws.send(JSON.stringify({
  type: 'chat',
  message: 'Hello everyone!'
}))
```

```
5. ws.on('message', (data) => {
  // Receives chat message
})

6. handleChatMessage(ws, msg)
  - Validate user is in room
  - Add timestamp
  - Get sender username

7. broadcastToRoom(room, {
  type: 'chat',
  username: 'Alice',
  message: 'Hello everyone!',
  timestamp: '2024-01-15T10:30:00'
})
```

CLIENT (Alice)

- 8. `ws.addEventListener('message')`
- 9. `handleChatMessage(data)`
 - Display message (own-message style)

CLIENT (Bob)

- 10. `ws.addEventListener('message')`
- 11. `handleChatMessage(data)`
 - Display message (other-message style)

CLIENT (Charlie)

- 12. `ws.addEventListener('message')`

13. handleChatMessage(data)
- Display message (other-message style)

Flow 4: User Leaves Room

LEAVE ROOM FLOW

CLIENT (Alice)

SERVER

1. User clicks "Leave Room"

2. sendMessage('leave-room', {
 room: 'general'
}))

3. ws.send(JSON.stringify({
 type: 'leave-room',
 room: 'general'
})))

4. handleLeaveRoom(ws, msg)
- Remove from rooms Map
- Clear user's current room

5. ws.send(JSON.stringify({
 type: 'left-room',
 room: 'general'
})))

6. handleLeftRoom(data)
- Clear messages
- Update UI
- Show "No room"

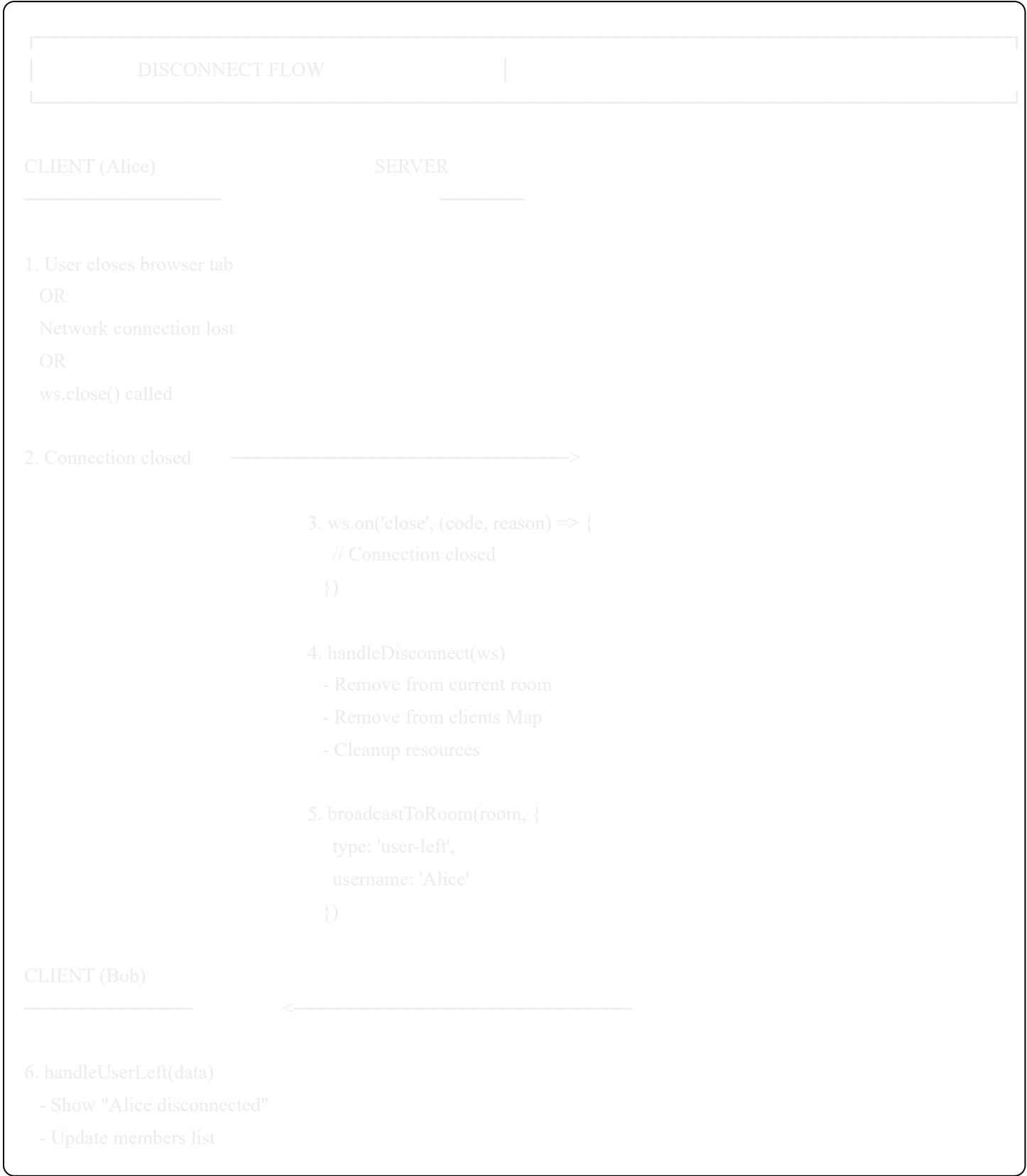
7. broadcastToRoom('general', {
 type: 'user-left',
 username: 'Alice'
}))

CLIENT (Bob - still in room)

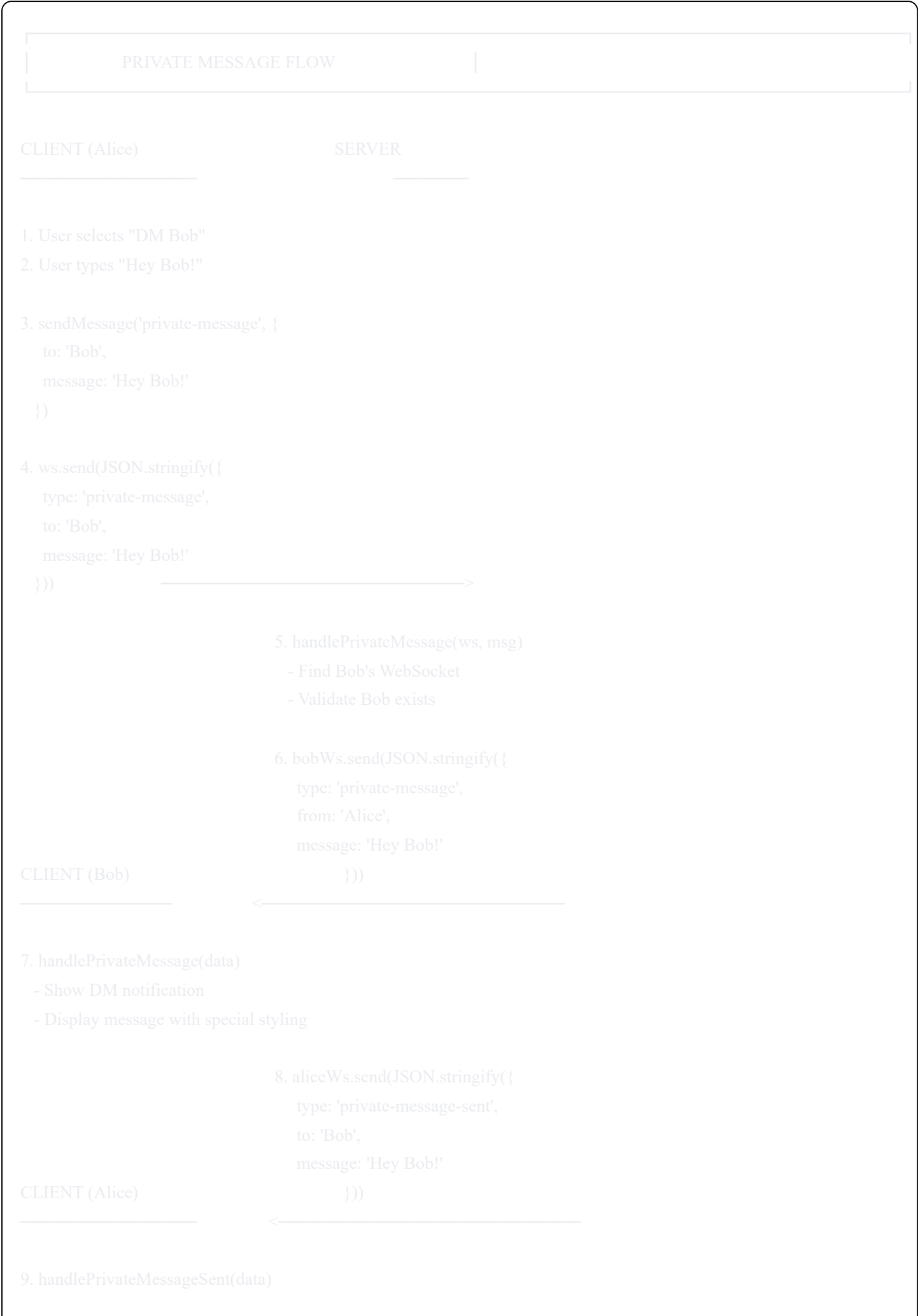


8. `handleUserLeft(data)`
- Show "Alice left the room"

Flow 5: User Disconnects



Flow 6: Private Message



- Confirm message sent
- Display in sent messages

Flow 7: Typing Indicator

TYPING INDICATOR FLOW

CLIENT (Alice)

SERVER

1. User starts typing in input
2. Input event fires

```
3. sendMessage('typing', {  
  isTyping: true  
})
```

```
4. ws.send(JSON.stringify({  
  type: 'typing',  
  isTyping: true  
}))
```

```
5. handleTyping(ws, msg)  
  - Get user's room  
  - Don't send to self
```

```
6. broadcastToRoom(room, {  
  type: 'typing',  
  username: 'Alice',  
  isTyping: true  
, excludeAlice)
```

CLIENT (Bob)

```
7. handleTyping(data)  
  - Show "Alice is typing..."
```

... 1 second of no typing ...

CLIENT (Alice)

```
8. ws.send(JSON.stringify({
  type: 'typing',
  isTyping: false
})))
```

```
9. broadcastToRoom(...)
```

CLIENT (Bob)

```
10. handleTyping(data)
    - Hide "Alice is typing..."
```

Flow 8: Create New Room

CREATE ROOM FLOW

CLIENT (Alice)

SERVER

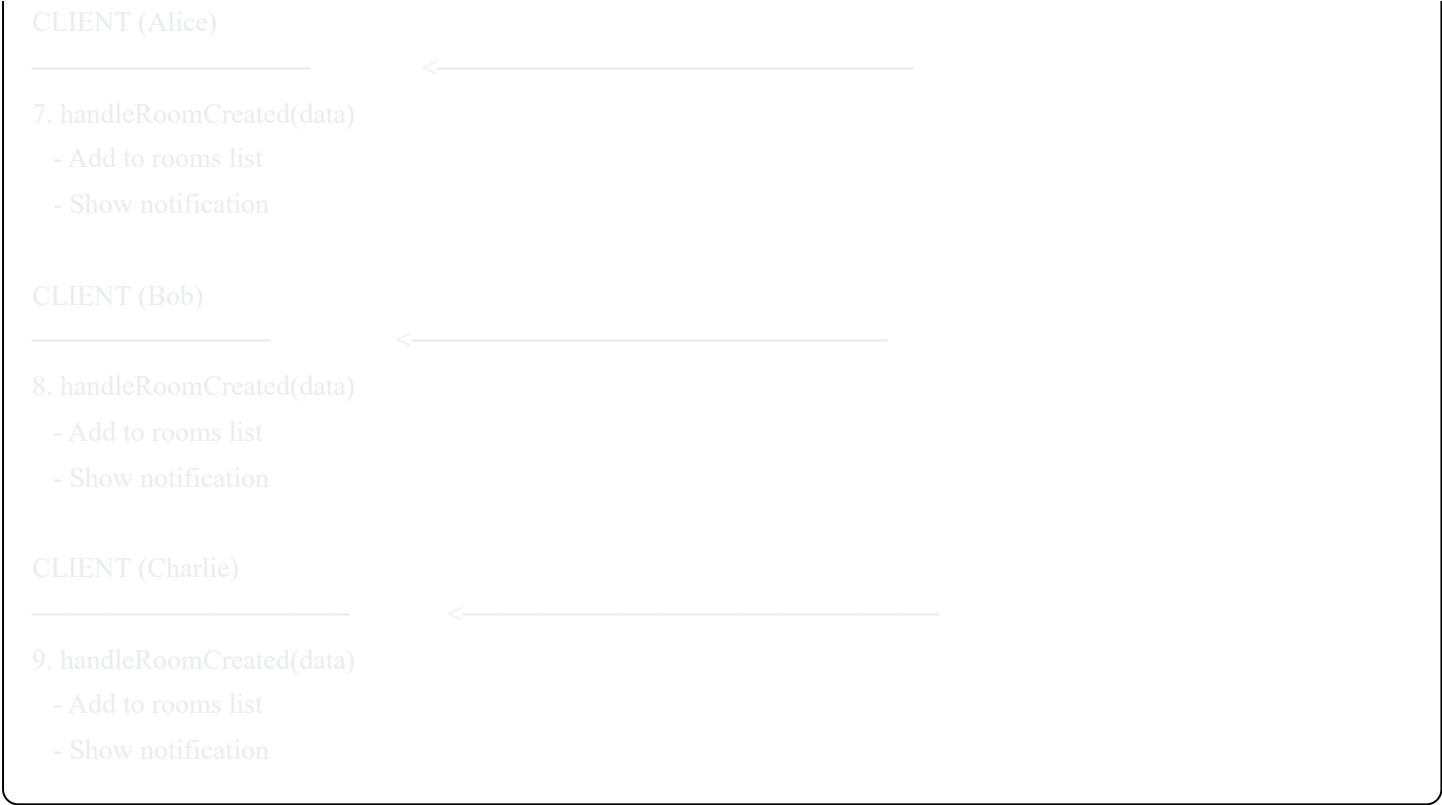
- 1. User clicks "Create Room"
- 2. User enters "project-alpha"

```
3. sendMessage('create-room', {
  room: 'project-alpha'
}))
```

```
4. ws.send(JSON.stringify({
  type: 'create-room',
  room: 'project-alpha'
})))
```

```
5. handleCreateRoom(ws, msg)
    - Validate room name
    - Check if exists
    - Create new room in Map
```

```
6. broadcast({
  type: 'room-created',
  room: 'project-alpha',
  creator: 'Alice'
})
```



4. Complete Event Reference Table

Native WebSocket Events

Client Event	Triggered By	Server Event	Triggered By
<code>open</code>	Server accepts connection	<code>connection</code>	Client connects
<code>message</code>	Server sends data	<code>message</code>	Client sends data
<code>close</code>	Server closes or network fails	<code>close</code>	Client closes or network fails
<code>error</code>	Connection/protocol error	<code>error</code>	Connection/protocol error
N/A (auto-handled)	Server sends ping	<code>ping</code>	Client sends ping
N/A (auto-handled)	Server sends pong	<code>pong</code>	Client sends pong

Application Message Types

Message Type	Direction	Purpose	Triggers On	Response Type
<code>set-username</code>	Client → Server	Set display name	User input	<code>username-set</code> or <code>error</code>

Message Type	Direction	Purpose	Triggers On	Response Type
<code>username-set</code>	Server → Client	Confirm username	Username validated	UI update
<code>join-room</code>	Client → Server	Join chat room	User action	<code>joined-room</code> + <code>user-joined</code> broadcast
<code>joined-room</code>	Server → Client	Confirm room join	Added to room	UI update
<code>user-joined</code>	Server → Broadcast	Notify others	User joins room	Display notification
<code>leave-room</code>	Client → Server	Leave chat room	User action	<code>left-room</code> + <code>user-left</code> broadcast
<code>left-room</code>	Server → Client	Confirm room leave	Removed from room	UI update
<code>user-left</code>	Server → Broadcast	Notify others	User leaves room	Display notification
<code>chat</code>	Client → Server	Send message	User sends	Broadcast to room
<code>chat</code>	Server → Broadcast	Relay message	Message received	Display message
<code>create-room</code>	Client → Server	Create new room	User action	<code>room-created</code> broadcast
<code>room-created</code>	Server → Broadcast	Notify of new room	Room created	Add to room list
<code>private-message</code>	Client → Server	Send DM	User action	Send to recipient
<code>private-message</code>	Server → Client	Receive DM	DM sent to you	Display DM
<code>private-message-sent</code>	Server → Client	Confirm DM sent	DM delivered	Show confirmation
<code>typing</code>	Client → Server	Typing status	User typing	Broadcast to room
<code>typing</code>	Server → Broadcast	Show typing	Someone typing	Show indicator
<code>room-members</code>	Server → Client	Members list	Join room	Display members
<code>get-rooms</code>	Client → Server	Request rooms	User action	<code>rooms-list</code>
<code>rooms-list</code>	Server → Client	Available rooms	Rooms requested	Display rooms

Message Type	Direction	Purpose	Triggers On	Response Type
get-users	Client → Server	Request users	User action	users-list
users-list	Server → Client	Online users	Users requested	Display users
error	Server → Client	Error message	Validation failed	Display error

5. Event Handler Code Templates

Server-Side Template

```
javascript
```



```
// Native WebSocket Events
wss.on('connection', (ws, request) => {
  console.log('Client connected');

  // Application message routing
  ws.on('message', (data) => {
    const message = JSON.parse(data.toString());

    switch (message.type) {
      case 'set-username':
        handleSetUsername(ws, message);
        break;
      case 'join-room':
        handleJoinRoom(ws, message);
        break;
      case 'leave-room':
        handleLeaveRoom(ws, message);
        break;
      case 'chat':
        handleChatMessage(ws, message);
        break;
      case 'create-room':
        handleCreateRoom(ws, message);
        break;
      case 'private-message':
        handlePrivateMessage(ws, message);
        break;
      case 'typing':
        handleTyping(ws, message);
        break;
      case 'get-rooms':
        handleGetRooms(ws, message);
        break;
      case 'get-users':
        handleGetUsers(ws, message);
        break;
      default:
        ws.send(JSON.stringify({ type: 'error', message: 'Unknown type' }));
    }
  });

  ws.on('close', (code, reason) => {
    console.log('Client disconnected');
    handleDisconnect(ws);
  });
});
```

```
ws.on('error', (error) => {  
  console.error('WebSocket error:', error);  
});  
  
ws.on('ping', () => {  
  console.log('Ping received');  
});  
  
ws.on('pong', () => {  
  console.log('Pong received');  
});  
});
```

Client-Side Template

javascript

```
// Create connection
const ws = new WebSocket('ws://localhost:3000');

// Native WebSocket Events
ws.addEventListener('open', (event) => {
  console.log('Connected to server');
});

// Application message routing
ws.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);

  switch (data.type) {
    case 'connection':
      handleConnection(data);
      break;
    case 'username-set':
      handleUsernameSet(data);
      break;
    case 'joined-room':
      handleJoinedRoom(data);
      break;
    case 'left-room':
      handleLeftRoom(data);
      break;
    case 'chat':
      handleChatMessage(data);
      break;
    case 'user-joined':
      handleUserJoined(data);
      break;
    case 'user-left':
      handleUserLeft(data);
      break;
    case 'room-members':
      handleRoomMembers(data);
      break;
    case 'room-created':
      handleRoomCreated(data);
      break;
    case 'private-message':
      handlePrivateMessage(data);
      break;
    case 'private-message-sent':
      handlePrivateMessageSent(data);
      break;
  }
});
```

```

    case 'typing':
      handleTyping(data);
      break;
    case 'rooms-list':
      handleRoomsList(data);
      break;
    case 'users-list':
      handleUsersList(data);
      break;
    case 'error':
      handleError(data);
      break;
    default:
      console.warn('Unknown message type:', data.type);
  }
});

ws.addEventListener('close', (event) => {
  console.log('Disconnected from server');
  console.log('Code:', event.code);
  console.log('Reason:', event.reason);
});

ws.addEventListener('error', (event) => {
  console.error('WebSocket error:', event);
});

// Send functions
function sendMessage(type, payload) {
  if (ws.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify({ type, ...payload }));
  }
}

```

6. Event Lifecycle Summary

Complete Request-Response Pattern

```

USER ACTION
  ↓
CLIENT EVENT HANDLER (onClick, onSubmit, etc.)
  ↓
PREPARE MESSAGE OBJECT { type: 'xxx', ...data }
  ↓

```

```
ws.send(JSON.stringify(message))
↓
WEBSOCKET PROTOCOL (frames the data)
↓
NETWORK (TCP)
↓
SERVER RECEIVES FRAME
↓
ws.on('message') FIRES
↓
JSON.parse(data)
↓
SWITCH ON message.type
↓
CALL APPROPRIATE HANDLER
↓
PROCESS BUSINESS LOGIC
↓
PREPARE RESPONSE { type: 'yyy', ...data }
↓
ws.send(JSON.stringify(response)) OR broadcast()
↓
WEBSOCKET PROTOCOL (frames the data)
↓
NETWORK (TCP)
↓
CLIENT(S) RECEIVE FRAME
↓
ws.addEventListener('message') FIRES
↓
JSON.parse(event.data)
↓
SWITCH ON data.type
↓
CALL APPROPRIATE HANDLER
↓
UPDATE UI
```

7. Quick Reference: Common Patterns

Pattern 1: Simple Request-Response

```
javascript
```

```
// CLIENT
ws.send(JSON.stringify({ type: 'get-rooms' }));

// SERVER
ws.on('message', (data) => {
  if (message.type === 'get-rooms') {
    ws.send(JSON.stringify({
      type: 'rooms-list',
      rooms: Array.from(rooms.keys())
    }));
  }
});

// CLIENT
ws.addEventListener('message', (e) => {
  const data = JSON.parse(e.data);
  if (data.type === 'rooms-list') {
    displayRooms(data.rooms);
  }
});
```

Pattern 2: Broadcast to All

```
javascript
```

```
// CLIENT A
ws.send(JSON.stringify({ type: 'create-room', room: 'new-room' }));

// SERVER
ws.on('message', (data) => {
  if (message.type === 'create-room') {
    rooms.set(message.room, new Set());

    // Send to ALL clients
    wss.clients.forEach((client) => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(JSON.stringify({
          type: 'room-created',
          room: message.room
        }));
      }
    });
  }
});

// ALL CLIENTS (A, B, C, etc.)
ws.addEventListener('message', (e) => {
  const data = JSON.parse(e.data);
  if (data.type === 'room-created') {
    addRoomToList(data.room);
  }
});
```

Pattern 3: Targeted Broadcast (Room-Specific)

javascript

```

// CLIENT A (in room 'general')
ws.send(JSON.stringify({
  type: 'chat',
  message: 'Hello room!'
}));

// SERVER
ws.on('message', (data) => {
  if (message.type === 'chat') {
    const clientInfo = clients.get(ws);
    const room = clientInfo.currentRoom;

    // Send to all clients in the SAME ROOM
    rooms.get(room).forEach((clientWs) => {
      if (clientWs.readyState === WebSocket.OPEN) {
        clientWs.send(JSON.stringify({
          type: 'chat',
          username: clientInfo.username,
          message: message.message
        }));
      }
    });
  }
});

// CLIENT B (in 'general') - RECEIVES
// CLIENT C (in 'tech') - DOES NOT RECEIVE
ws.addEventListener('message', (e) => {
  const data = JSON.parse(e.data);
  if (data.type === 'chat') {
    displayMessage(data.username, data.message);
  }
});

```

Pattern 4: Private/Direct Message

javascript


```

// CLIENT A
ws.send(JSON.stringify({
  type: 'private-message',
  to: 'Bob',
  message: 'Secret message'
}));

// SERVER
ws.on('message', (data) => {
  if (message.type === 'private-message') {
    // Find Bob's socket
    let targetWs = null;
    for (const [socket, info] of clients.entries()) {
      if (info.username === message.to) {
        targetWs = socket;
        break;
      }
    }

    if (targetWs) {
      // Send ONLY to Bob
      targetWs.send(JSON.stringify({
        type: 'private-message',
        from: clients.get(ws).username,
        message: message.message
      }));
    }
  }
});

// CLIENT B (Bob) - RECEIVES
// CLIENT C, D, E - DO NOT RECEIVE
ws.addEventListener('message', (e) => {
  const data = JSON.parse(e.data);
  if (data.type === 'private-message') {
    displayPrivateMessage(data.from, data.message);
  }
});

```

Summary

Key Takeaways

1. **Native Events** (`open`, `message`, `close`, `error`, `ping`, `pong`) are built into WebSocket protocol

2. **Application Events** are custom types we create using JSON with a `type` field
3. All custom messages flow through the single `message` event
4. Server routes messages based on `type` field to appropriate handlers
5. Client routes responses based on `type` field to appropriate handlers
6. Broadcast patterns determine who receives what messages (all, room, or specific user)

The Golden Rule

WebSocket has ONE event for all messages: 'message'

We create MANY logical event types using: { type: 'event-name', ...data }